EXPERIMENT: CRYPT-ARITHMETIC PROBLEM

Aim

To write a Python program to solve the Crypt-Arithmetic problem (SEND + MORE = MONEY) using Artificial Intelligence techniques.

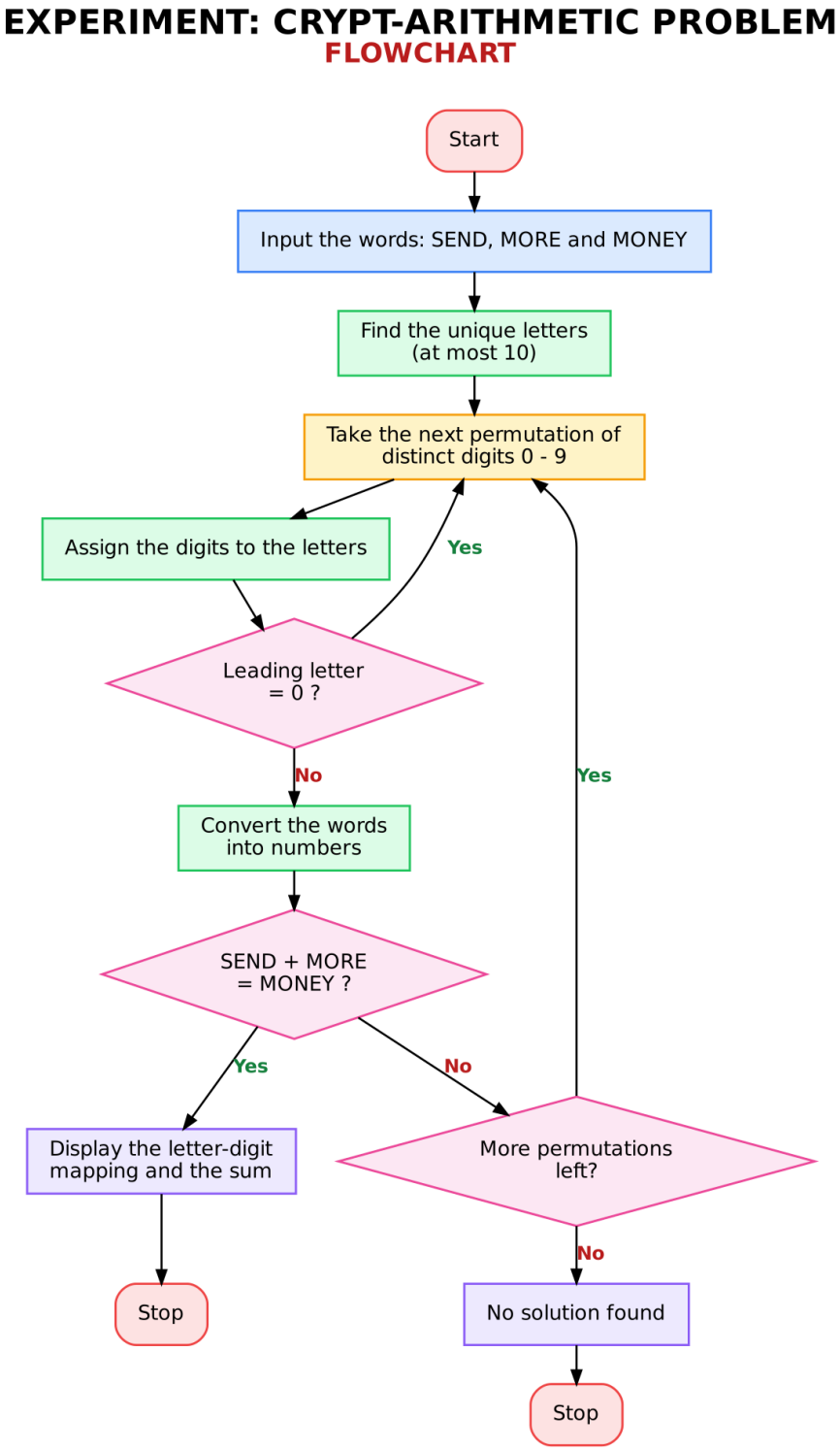
Objective

- To understand crypt-arithmetic puzzles as constraint satisfaction problems.
- To implement the generate-and-test technique using permutations of digits.
- To assign a unique digit to every letter so that the arithmetic equation becomes true.

Algorithm

1. Start.
2. Read the three words (SEND, MORE and MONEY).
3. Find the unique letters used in all the words (maximum 10).
4. Generate a permutation of distinct digits from 0 to 9 for these letters.
5. Assign the digits to the letters.
6. If the first letter of any word is assigned 0, reject the assignment.
7. Convert the words into numbers and check whether SEND + MORE = MONEY.
8. If the equation is true, display the mapping and the sum, then stop.
9. Otherwise, try the next permutation.
10. If all permutations have been tried, display that no solution exists. Stop.

Flowchart:



Code (Python)

```
from itertools import permutations

def solve_cryptarithmetic(word1, word2, result):
    letters = sorted(set(word1 + word2 + result))
    if len(letters) > 10:
        return None
    first_letters = {word1[0], word2[0], result[0]}
```

```
    def to_number(word, mapping):
        num = 0
        for ch in word:
            num = num * 10 + mapping[ch]
        return num

    for digits in permutations(range(10), len(letters)):
        mapping = dict(zip(letters, digits))

        # a word cannot start with 0
        if any(mapping[ch] == 0 for ch in first_letters):
            continue

        if to_number(word1, mapping) + to_number(word2, mapping) == to_number(result, mapping):
            return mapping
    return None

word1, word2, result = "SEND", "MORE", "MONEY"
solution = solve_cryptarithmetic(word1, word2, result)

if solution:
    print("Letter to digit mapping:")
    for letter in sorted(solution):
        print(letter, "=", solution[letter])

    n1 = int("".join(str(solution[c]) for c in word1))
    n2 = int("".join(str(solution[c]) for c in word2))
    n3 = int("".join(str(solution[c]) for c in result))
    print()
    print(" ", n1)
    print("+", n2)
    print("-------")
    print(" ", n3)
else:
    print("No solution found")
```

Output

Python Shell - Output

```
Letter to digit mapping:
D = 7
E = 5
M = 1
N = 6
O = 0
R = 8
S = 9
Y = 2

  9567
+ 1085
-------
  10652
```

Output Result

Thus, the Python program to solve the Crypt-Arithmetic problem was executed successfully. The solution obtained is S=9, E=5, N=6, D=7, M=1, O=0, R=8, Y=2, i.e. 9567 + 1085 = 10652.

Conclusion

The experiment helped in understanding constraint satisfaction problems. The Crypt-Arithmetic puzzle was solved successfully by testing digit assignments until the equation and the constraints were satisfied.